

KANBIEN ARCHITECTURE GUIDE

Platform Layer Architecture

A first-principles design for structuring `/platform` in a serious SaaS monorepo.

Prepared from the architecture response. Generated June 21, 2026.

Table of Contents

The clean mental model	4
1. What a platform layer is	5
2. Why it should be separate from apps, features, and core packages	6
Apps change because the business changes.....	6
Platform changes because the runtime model changes	6
Core packages change because shared primitives evolve.....	6
3. What problems <code>/platform</code> solves	7
Consistent startup	7
Consistent request handling	7
Predictable app integration.....	7
Operational readiness.....	7
Security consistency	7
Testability.....	8
4. What problems <code>/platform</code> should not solve	9
5. How <code>/platform</code> differs from backend business logic.....	10
6. How <code>/platform</code> differs from infrastructure	11
7. How <code>/platform</code> differs from shared core libraries.....	12
8. When a platform layer becomes over-engineered	13
Recommended Kanbien repo model	14
Proposed <code>/platform</code> structure	15
How <code>/platform</code> should handle each runtime concern.....	17
HTTP server startup	17
App mounting	17
Route registration	18
Middleware	18
CORS.....	19
Rate limiting	20
Auth hooks.....	20
Request context.....	21
Error handling	22
Logging	22
Health checks	23
Environment config.....	23
Feature flags	24
API versioning	25
Background workers	25

Graceful shutdown	26
Folder-by-folder design	27
platform/contracts/	27
platform/config/	28
platform/runtime/	30
platform/server/	31
platform/security/	33
platform/observability/	34
platform/health/	35
platform/workers/	36
platform/testing/	38
The business app mounting contract.....	39
Example platform contract	40
Worked example: apps/crm/app.mount.ts	43
Example CRM permission file	46
Example CRM route handler	47
Example CRM jobs	49
Example CRM health checks.....	50
platform/server/mount.ts	51
platform/server/registry.ts	53
How route registration becomes real HTTP routes.....	56
What should remain inside apps/crm	58
What should move into packages/core	59
What belongs in /platform.....	61
Minimal v1 platform architecture.....	62
Must-have v1 pieces	62
Nice-to-have later	62
The simple v1 runtime flow	64
The most important design rule	65

The clean mental model

For Kanbien, treat `/platform` as the **runtime composition layer**.

It is the part of the codebase that answers:

"How does Kanbien start, accept traffic, authenticate requests, route work to business apps, run background jobs, log, expose health, and shut down safely?"

It should **not** answer:

"What is a CRM user? Who can edit a customer record? What does onboarding mean? How should billing calculate usage?"

That distinction matters. A good platform makes business apps easier to build. A bad platform becomes a second product that every feature team has to fight.

A useful dependency picture is:

```
infra/      creates cloud resources
  v
platform/   runs the application process and provides runtime rules
  v
apps/       CRM, billing, admin, onboarding, etc.
  v
packages/   reusable pure/shared libraries
```

More precisely:

```
apps/* -----> platform/contracts
apps/* -----> packages/*
platform/runtime -> packages/core

platform/server/mount.ts -> apps/*/app.mount.ts
```

That last line is the important exception: the **composition root** imports apps so it can mount them. The rest of `/platform` should not know business apps exist.

1. What a platform layer is

A platform layer is the **host environment** for your business apps.

It owns the common runtime machinery:

```
process startup
HTTP server
route registration
middleware
auth integration
request context
error mapping
logging
health checks
config loading
feature flag access
job execution
shutdown
```

In a backend SaaS product, /platform is like the operating system for your product backend. Apps plug into it. The platform gives them common rules and services.

The platform should say:

"Here is how an app registers routes, permissions, health checks, and jobs."

It should not say:

"Here is how CRM calculates a user's sales territory."

2. Why it should be separate from apps, features, and core packages

Because those layers change for different reasons.

Apps change because the business changes

`apps/crm` changes when you add CRM features:

```
users
accounts
contacts
pipelines
permissions
CRM workflows
```

Platform changes because the runtime model changes

`platform` changes when you alter how Kanbien operates:

```
auth provider
logging format
rate limiting
HTTP framework
worker runtime
health checks
request context
shutdown behavior
```

Core packages change because shared primitives evolve

`packages/core` should contain reusable, framework-independent primitives:

```
Result
EntityId
TenantId
Money
EmailAddress
Pagination
DomainError
Clock
```

It should not start servers, connect queues, or register routes.

Separating these avoids the classic monorepo failure mode: everything imports everything, and every change becomes risky.

3. What problems /platform solves

A strong platform layer solves these problems:

Consistent startup

Every environment starts the same way:

```
load env
validate config
create logger
create runtime resources
mount apps
register routes/jobs/health checks
start server or worker
install shutdown handlers
```

Consistent request handling

Every request gets the same treatment:

```
request id
correlation id
tenant detection
auth parsing
rate limiting
logging
error mapping
response shape
```

Predictable app integration

Apps do not invent their own way to expose routes or background jobs. They use one contract.

Operational readiness

The platform makes Kanbien observable and manageable:

```
/livez
/readyz
structured logs
metrics
tracing hooks
graceful shutdown
job retries
```

Security consistency

CORS, auth hooks, permissions, and rate limiting are not reinvented by every app.

Testability

You can mount a fake app into the platform and test the whole backend shape without running the full product.

4. What problems /platform should not solve

The platform should **not** solve business problems.

It should not contain:

```
CRM user lifecycle rules
billing calculations
sales pipeline rules
customer onboarding logic
feature-specific workflows
domain-specific repository methods
business-specific permission decisions
business-specific quotas
```

It should also avoid becoming a dumping ground for "shared stuff."

Bad platform code looks like this:

```
platform/crm-user-service.ts
platform/billing-plan-rules.ts
platform/customer-status-machine.ts
platform/salesforce-sync.ts
```

Those belong in apps or business packages, not platform.

The platform provides **mechanisms**. Apps provide **meaning**.

For example:

```
Platform mechanism:
  "This route requires permission crm.users.read."

CRM meaning:
  "A regional sales manager may see users in their assigned region."
```

The first can live in platform contracts. The second belongs to `apps/crm`.

5. How `/platform` differs from backend business logic

Backend business logic answers product questions.

Platform logic answers runtime questions.

Example: request to list CRM users.

```
GET /api/v1/crm/users
```

The platform handles:

```
HTTP parsing
route lookup
CORS
rate limit
auth token parsing
request context creation
permission check hook
logging
error handling
response serialization
```

The CRM app handles:

```
which users exist
which users are visible
what filters are allowed
how CRM roles work
which database queries to run
how CRM-specific errors behave
```

A good test:

Could this code still make sense if Kanbien had no CRM app?

If yes, it may belong in `/platform`.

Does this code mention leads, deals, accounts, campaigns, billing plans, invoices, or customer lifecycle?

Then it probably does **not** belong in `/platform`.

6. How `/platform` differs from infrastructure

Infrastructure is the external environment.

`infra/` creates and configures things like:

```
load balancers
databases
Redis
queues
object storage
DNS
secrets
Kubernetes/ECS/Fly/Render resources
networking
IAM
```

`platform/` is application code that runs inside the deployed process.

Example:

```
infra/
  creates Redis

platform/
  connects to Redis
  uses Redis for rate limiting
  uses Redis-backed queues
  closes Redis during shutdown
```

Infrastructure says:

```
"There is a database."
```

Platform says:

```
"Here is the database client lifecycle."
```

Apps say:

```
"Here is the CRM repository query."
```

Do not put Terraform, Pulumi, Kubernetes manifests, or cloud provisioning code inside `/platform`.

7. How `/platform` differs from shared core libraries

`packages/core` should be mostly pure TypeScript. It should have little to no runtime infrastructure knowledge.

Good `packages/core` examples:

```
packages/core/result  
packages/core/errors  
packages/core/id  
packages/core/time  
packages/core/pagination  
packages/core/tenant  
packages/core/auth-types
```

Bad `packages/core` examples:

```
packages/core/http-server.ts  
packages/core/redis-client.ts  
packages/core/fastify-plugin.ts  
packages/core/crm-user-service.ts  
packages/core/background-worker.ts
```

A useful rule:

`packages/core` should be usable in a CLI script, web app, worker, or unit test without starting the Kanbien backend.

The platform may depend on `packages/core`.

`packages/core` should not depend on `/platform`.

8. When a platform layer becomes over-engineered

A platform becomes over-engineered when it creates more friction than leverage.

Warning signs:

- You need to change platform code for every normal product feature.
- Apps cannot register a simple route without learning an internal framework.
- The platform has abstractions for databases, queues, events, permissions, and plugins that Kanbien does not actually need yet.
- The platform hides the HTTP framework so aggressively that debugging becomes harder.
- There is a generic workflow engine before there are real workflows.
- There is a plugin marketplace before there are three real apps.
- There are ten configuration layers.
- Business engineers constantly need platform engineers to ship features.
- The platform contains domain words like CRM, invoice, deal, lead, account, subscription, or campaign.

The healthy version is boring:

- one way to start
- one way to mount apps
- one way to register routes
- one way to run jobs
- one way to log
- one way to handle errors
- one way to shut down

Do that first. Earn every abstraction after that.

Recommended Kanbien repo model

Given:

```
kanbien/  
  platform/  
  apps/  
  packages/  
  harness/  
  tools/  
  infra/
```

I would define the top-level responsibilities like this:

```
platform/  
  Runtime host for backend apps.  
  
apps/  
  Business applications and bounded contexts:  
  crm, billing, admin, identity, onboarding, etc.  
  
packages/  
  Reusable libraries:  
  core primitives, clients, shared UI, shared validation, SDKs.  
  
harness/  
  Test and local-development compositions:  
  fake apps, seeded environments, integration harnesses, contract tests.  
  
tools/  
  Developer/build tools:  
  code generators, lint rules, repo scripts, migration runners.  
  
infra/  
  Infrastructure-as-code:  
  cloud resources, networking, databases, queues, deployment definitions.
```

The platform should be the backend runtime host, not a business app and not cloud infrastructure.

Proposed /platform structure

A serious but not bloated v1:

```
platform/  
  README.md  
  
  contracts/  
    app.ts  
    route.ts  
    permission.ts  
    job.ts  
    health.ts  
    context.ts  
    config.ts  
  
  config/  
    env.ts  
    schema.ts  
    app-config.ts  
  
  runtime/  
    context.ts  
    resources.ts  
    errors.ts  
    flags.ts  
    lifecycle.ts  
  
  server/  
    main.ts  
    create-server.ts  
    mount.ts  
    registry.ts  
    routes.ts  
    middleware.ts  
    versioning.ts  
  
  security/  
    auth-hooks.ts  
    authorization.ts  
    cors.ts  
    rate-limit.ts  
  
  observability/  
    logger.ts  
    request-logging.ts  
    metrics.ts  
    tracing.ts  
  
  health/  
    registry.ts  
    routes.ts  
    checks.ts  
  
  workers/  
    main.ts  
    registry.ts  
    queue.ts  
    scheduler.ts  
    processor.ts  
  
  testing/  
    fake-registry.ts  
    fake-context.ts  
    test-server.ts  
    mount-contract.ts
```

This gives Kanbien a clean runtime shape without building a giant internal framework.

How /platform should handle each runtime concern

HTTP server startup

The platform should own process startup.

Startup should be boring and deterministic:

1. Load raw environment variables.
2. Validate environment config.
3. Create root logger.
4. Create runtime resources.
5. Create platform registries.
6. Mount apps.
7. Validate registrations.
8. Create HTTP server.
9. Register global middleware.
10. Register health routes.
11. Register app routes.
12. Start listening.
13. Install graceful shutdown handlers.

Business apps should never call:

```
server.listen(...)
app.listen(...)
process.on('SIGTERM', ...)
```

Those are platform concerns.

App mounting

Apps should mount through a predictable contract.

The platform gives an app a registry. The app contributes:

```
routes
permissions
background jobs
health checks
optional config schema
optional lifecycle hooks
```

The app does **not** receive the raw HTTP server.

This is important. If you pass a Fastify or Express instance directly into every app, you couple every app to your framework forever.

Better:

```
crmApp.mount(registry, deps);
```

The platform later translates registered routes into Fastify, Express, Hono, or whatever you choose internally.

Route registration

Routes should be registered declaratively.

An app should say:

```
method
path
API version
auth requirement
permissions
rate limit policy
request schema
response schema
handler
```

The platform should validate:

```
no duplicate routes
all permissions exist
all app routes are namespaced
all public routes are explicitly public
no app can steal /health, /metrics, /admin unless allowed
```

Recommended external route shape:

```
/api/v1/crm/users
/api/v1/crm/users/:userId
```

The app should register local paths like:

```
/users
/users/:userId
```

The platform adds:

```
/api/v1
/crm
```

Middleware

The platform owns middleware order.

A good default order:

```
request id
correlation id
request logging
CORS / preflight
security headers
body parsing
rate limiting
auth parsing
request context creation
authorization
route validation
handler execution
error mapping
response logging
```

Apps should not freely install global middleware.

They may declare route-level policies:

```
auth: 'required'
permissions: ['crm.users.read']
rateLimit: 'standard'
```

But they should not do this:

```
server.use(...)
fastify.addHook(...)
```

That leaks the platform's internals into business code.

CORS

CORS should be centrally configured.

The platform should own:

```
allowed origins
allowed methods
allowed headers
credential policy
preflight behavior
environment-specific differences
```

Apps may declare that a route is public, but they should not independently decide allowed origins.

Example:

```
production:
  https://app.kanbien.com

staging:
  https://staging.kanbien.com
```

```
local:
  http://localhost:3000
```

Rate limiting

The platform should enforce rate limiting.

It should support:

```
global default limits
stricter limits for sensitive routes
public route limits
authenticated user limits
tenant-level limits
IP fallback limits
```

Apps can request a policy:

```
rateLimit: {
  policy: 'standard'
}
```

Or:

```
rateLimit: {
  policy: 'sensitive',
  key: 'principal'
}
```

But product-specific quotas should stay in the business app.

For example:

```
Platform rate limit:
  "100 requests per minute per user."

CRM business quota:
  "This customer plan allows 5,000 CRM exports per month."
```

The second belongs in CRM or billing, not platform.

Auth hooks

The platform should authenticate requests.

It should know how to:

```
read Authorization headers
verify sessions or JWTs
load identity claims
create a Principal object
```

```
attach principal to request context
reject invalid credentials
```

The platform should not contain CRM-specific authorization rules.

A good split:

```
Platform:
  "This request came from user_123 in tenant_456."

Platform:
  "This route requires crm.users.read."

CRM:
  "This user can only see CRM users in their region."
```

So the platform handles authentication and coarse permission enforcement.

Apps still handle resource-specific business authorization.

Request context

Every handler should receive a `RequestContext`.

It should contain things like:

```
type RequestContext = {
  requestId: string;
  correlationId: string;
  tenantId?: string;
  principal?: Principal;
  logger: Logger;
  flags: FeatureFlagReader;
  config: AppConfigReader;
  now: () => Date;
  abortSignal?: AbortSignal;
};
```

The request context gives handlers the common runtime facts.

It should not become a junk drawer.

Avoid this:

```
ctx.crmUserService
ctx.billingService
ctx.everyRepository
ctx.everything
```

Business dependencies should be constructed inside apps and injected into app handlers intentionally.

Error handling

The platform should map errors to consistent API responses.

For example:

```
ValidationError -> 400
Unauthorized    -> 401
Forbidden       -> 403
NotFound        -> 404
Conflict        -> 409
RateLimited     -> 429
Unexpected      -> 500
```

Response shape should be consistent:

```
{
  "error": {
    "code": "not_found",
    "message": "User not found",
    "requestId": "req_123"
  }
}
```

Apps should not hand-roll HTTP error responses everywhere.

But the platform should not know every CRM domain error.

A clean app route handler can translate domain errors at the edge:

```
if (result.error === 'CRM_USER_NOT_FOUND') {
  throw notFound('User not found');
}
```

The platform maps `notFound` to HTTP `404`.

Logging

The platform owns the logger.

Use structured logs, not random strings.

Good:

```
{
  "level": "info",
  "msg": "request completed",
  "requestId": "req_123",
  "tenantId": "tenant_456",
  "principalId": "user_789",
  "method": "GET",
  "path": "/api/v1/crm/users",
  "statusCode": 200,
  "durationMs": 24
}
```

Apps should log through `ctx.logger`.

They should not create their own logging frameworks.

Good:

```
ctx.logger.info({ userId }, 'crm user updated');
```

Bad:

```
console.log('updated user', userId);
```

Health checks

The platform should expose standard health endpoints.

Recommended:

```
GET /livez
  Is the process alive?
  Should not check database.

GET /readyz
  Is the process ready to serve traffic?
  Checks critical dependencies.

GET /healthz
  Human/debug summary, optional.
```

Apps can register app-specific readiness checks:

```
crm.users.repository
crm.users.search-index
billing.stripe-connectivity
```

The platform aggregates them.

Important distinction:

```
Liveness:
  "Should the orchestrator restart me?"

Readiness:
  "Should the load balancer send me traffic?"
```

Do not put expensive business audits in health checks.

Environment config

The platform should load and validate environment config at startup.

Bad:

```
const port = process.env.PORT;
```

Good:

```
const config = loadPlatformConfig(process.env);
```

The platform should own global config:

```
NODE_ENV  
PORT  
LOG_LEVEL  
CORS_ALLOWED_ORIGINS  
AUTH_JWKS_URL  
REDIS_URL  
DATABASE_URL  
FEATURE_FLAGS_PROVIDER  
RATE_LIMIT_STORE_URL
```

Apps may own namespaced app config:

```
CRM_USER_SYNC_ENABLED  
CRM_USER_SYNC_INTERVAL_SECONDS  
CRM_EXPORT_MAX_ROWS
```

The platform can validate app config by allowing apps to register config schemas, but the meaning of app-specific config belongs to the app.

Feature flags

The platform should provide a consistent feature flag reader.

Apps should be able to do:

```
const enabled = await ctx.flags.isEnabled('crm.users.bulk-import');
```

The platform owns:

```
flag provider integration  
evaluation context  
fallback behavior  
timeouts  
logging  
local/dev flag overrides
```

Apps own:


```
flag names
what the flag means
what behavior changes
```

Do not build a feature flag platform in v1 unless you need one. Start with a simple provider abstraction that can be backed by config, a database, or an external flag service later.

API versioning

The platform should own the versioning strategy.

For a SaaS API, use path versioning first:

```
/api/v1/crm/users
/api/v2/crm/users
```

Apps register routes against explicit versions:

```
version: 'v1'
```

API version is not the same as app package version.

```
API version:
  External contract.

App version:
  Internal code release.
```

The platform should detect route collisions:

```
GET /api/v1/crm/users
GET /api/v1/crm/users
```

That should fail at startup.

Background workers

The platform should own worker process startup and job execution mechanics.

It should handle:

```
queue connection
job registry
payload validation
retry policy
dead-letter handling
scheduled jobs
job logging
job metrics
graceful worker shutdown
```

Apps register jobs:

```
crm.users.sync-from-identity  
crm.users.rebuild-search-index
```

Apps implement job behavior.

The platform runs the jobs.

In production, I would usually run API and worker as separate process modes:

```
node dist/platform/server/main.js  
node dist/platform/workers/main.js
```

Same app registry. Different runtime entrypoint.

Graceful shutdown

The platform should own shutdown.

On `SIGTERM` or `SIGINT`, it should:

```
mark readiness false  
stop accepting new HTTP requests  
allow in-flight requests to finish  
stop taking new jobs  
let active jobs finish or time out  
run app shutdown hooks  
close queues  
close database pools  
flush logs/metrics  
exit cleanly
```

Apps can register cleanup hooks, but they should not install their own process signal handlers.

Folder-by-folder design

platform/contracts/

Purpose

Defines the public TypeScript contract between business apps and the platform.

This is the most important folder. Apps should depend on this, not on platform internals.

Contents

```
app.ts
route.ts
permission.ts
job.ts
health.ts
context.ts
config.ts
```

Example files

```
// platform/contracts/app.ts
export interface PlatformApp {
  name: string;
  basePath: `/${string}`;
  mount(registry: AppRegistry, deps: AppMountDeps): void | Promise<void>;
}
```

```
// platform/contracts/route.ts
export type HttpMethod = 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE';
export type ApiVersion = 'v1' | 'v2';

export interface RouteDefinition {
  method: HttpMethod;
  version: ApiVersion;
  path: `/${string}`;
  auth: AuthPolicy;
  permissions?: string[];
  rateLimit?: RateLimitPolicy;
  handler: RouteHandler;
}
```

Allowed dependencies

```
TypeScript types
small pure packages from packages/core
schema interfaces
```

Forbidden dependencies

```
Fastify
Express
```

```
database clients
Redis
queues
apps/*
infra/*
business services
```

Ownership rules

The platform owner owns this contract. App teams can propose changes, but changes should be reviewed carefully because this is the boundary every app relies on.

Testing approach

Use contract tests:

```
fake app can mount
invalid route fails
duplicate permission fails
route without known permission fails
job without payload schema fails
```

Build approach

Export this as the stable public API:

```
@kanbien/platform/contracts
```

Apps should import only from this public contract path.

platform/config/

Purpose

Loads, validates, and exposes typed configuration.

Contents

```
env.ts
schema.ts
app-config.ts
```

Example files

```
// platform/config/env.ts
export function loadPlatformConfig(rawEnv: NodeJS.ProcessEnv): PlatformConfig {
  // validate once at startup
}
```

```
// platform/config/app-config.ts
export interface AppConfigReader {
```

```
getString(key: string): string;  
getNumber(key: string): number;  
getBoolean(key: string): boolean;  
optionalString(key: string): string | undefined;  
}
```

Allowed dependencies

```
env validation library  
packages/core primitive types  
platform/contracts config types
```

Forbidden dependencies

```
apps/*  
HTTP server  
business repositories  
cloud IaC code
```

Ownership rules

Platform owns global config. Apps own namespaced app config.

For example:

```
Platform owns:  
PORT  
LOG_LEVEL  
CORS_ALLOWED_ORIGINS  
AUTH_JWKS_URL  
  
CRM owns:  
CRM_USER_SYNC_ENABLED  
CRM_EXPORT_MAX_ROWS
```

Testing approach

Test config parsing heavily:

```
missing required env fails  
invalid port fails  
bad URL fails  
safe defaults work in local/dev  
secrets are not logged
```

Build approach

Compiled into both server and worker entrypoints.

platform/runtime/

Purpose

Contains runtime primitives that are not specifically HTTP, jobs, or auth.

Contents

```
context.ts
resources.ts
errors.ts
flags.ts
lifecycle.ts
```

Example files

```
// platform/runtime/context.ts
export function createRequestContext(input: CreateContextInput): RequestContext {
  return {
    requestId: input.requestId,
    correlationId: input.correlationId,
    tenantId: input.tenantId,
    principal: input.principal,
    logger: input.logger,
    flags: input.flags,
    config: input.config,
    now: () => new Date(),
    abortSignal: input.abortSignal
  };
}
```

```
// platform/runtime/resources.ts
export interface RuntimeResources {
  db?: unknown;
  cache?: unknown;
  queue?: unknown;
}
```

Allowed dependencies

```
platform/contracts
platform/config
platform/observability
technical clients such as database/cache/queue clients
```

Forbidden dependencies

```
apps/*
business repositories
business services
domain-specific workflows
```

Ownership rules

Platform owns runtime lifecycle. Apps may consume resources through typed interfaces but should not control process-level lifecycle.

Testing approach

Test context creation, flag fallback, resource creation, and lifecycle cleanup.

Build approach

Internal platform module. Not generally imported directly by apps except through approved contract types.

platform/server/

Purpose

Owns the HTTP API process.

Contents

```
main.ts
create-server.ts
mount.ts
registry.ts
routes.ts
middleware.ts
versioning.ts
```

Example files

```
main.ts
  executable entrypoint

create-server.ts
  creates HTTP server and applies platform middleware

mount.ts
  imports app mount files and mounts them into registries

registry.ts
  stores routes, permissions, jobs, health checks

routes.ts
  turns RouteDefinition objects into actual HTTP routes

middleware.ts
  defines middleware order

versioning.ts
  applies /api/v1, /api/v2 path rules
```

Allowed dependencies

```
HTTP framework
platform/contracts
platform/config
platform/security
platform/health
platform/observability
platform/runtime
apps/*/app.mount.ts only in mount.ts
```

Forbidden dependencies

```
apps/* internal feature files
CRM services
billing services
repositories
infra/*
```

The only acceptable business-app import is in the composition root:

```
// platform/server/mount.ts
import { crmApp } from '../../../../apps/crm/app.mount';
```

The platform should not import this:

```
import { UserService } from '../../../../apps/crm/users/user.service';
```

Ownership rules

Platform owns server startup, middleware ordering, route adaptation, and app composition.

App teams own their own `app.mount.ts` files.

Testing approach

Use integration tests:

```
server starts with fake app
registered route responds
unauthenticated request fails
permission failure returns 403
duplicate route fails startup
error maps to expected shape
```

Build approach

Build as an executable entrypoint:

```
dist/platform/server/main.js
```

This is what the API container/process runs.

platform/security/

Purpose

Owns cross-cutting security mechanisms.

Contents

```
auth-hooks.ts
authorization.ts
cors.ts
rate-limit.ts
```

Example files

```
// platform/security/auth-hooks.ts
export interface AuthHooks {
  authenticate(input: AuthenticateInput): Promise<Principal | undefined>;
}
```

```
// platform/security/authorization.ts
export async function authorizeRoute(
  principal: Principal | undefined,
  permissions: string[]
): Promise<void> {
  // coarse permission enforcement
}
```

Allowed dependencies

```
auth provider SDKs
JWT/session libraries
cache/rate-limit store clients
platform/contracts
platform/config
platform/observability
```

Forbidden dependencies

```
CRM-specific permission logic
billing plan logic
business quotas
apps/*
```

Ownership rules

Platform owns authentication, CORS, rate limit enforcement, and coarse permission checks.

Apps own business-specific authorization.

Testing approach

Test security behavior with fake identities:

```
missing auth returns 401
invalid token returns 401
valid token creates principal
missing permission returns 403
public route allows anonymous request
rate limit returns 429
CORS preflight works
```

Build approach

Internal platform module used by `server` and sometimes `workers`.

platform/observability/

Purpose

Provides logging, metrics, and tracing hooks.

Contents

```
logger.ts
request-logging.ts
metrics.ts
tracing.ts
```

Example files

```
// platform/observability/logger.ts
export function createLogger(config: LoggingConfig): Logger {
  // create structured logger
}
```

```
// platform/observability/request-logging.ts
export function logRequestCompleted(input: RequestLogInput): void {
  // standard request-completed event
}
```

Allowed dependencies

```
logging libraries
metrics libraries
OpenTelemetry-style libraries
platform/config
platform/contracts
```

Forbidden dependencies

```
apps/*
business logic
repositories
feature-specific analytics
```

Ownership rules

Platform owns log format and common telemetry.

Apps own meaningful business events, but they should emit them through platform-provided logging/metrics interfaces.

Testing approach

Test that logs contain:

```
requestId
correlationId
tenantId where available
status code
duration
safe error code
no secrets
```

Build approach

Compiled into server and worker entrypoints.

platform/health/

Purpose

Aggregates platform and app health checks.

Contents

```
registry.ts
routes.ts
checks.ts
```

Example files

```
// platform/health/registry.ts
export interface HealthRegistry {
  register(check: HealthCheckDefinition): void;
}
```

```
// platform/health/routes.ts
export function registerHealthRoutes(server: HttpServer, registry: HealthRegistry): void {
```

```
// /livez, /readyz, /healthz  
}
```

Allowed dependencies

```
platform/contracts  
platform/runtime resources  
platform/observability
```

Forbidden dependencies

```
business workflows  
expensive data audits  
apps/* internals
```

Ownership rules

Platform owns health endpoints and aggregation.

Apps own their own health check definitions.

Testing approach

Test:

```
/livez succeeds without dependency checks  
/readyz fails if critical dependency fails  
app health checks are included  
slow checks timeout  
health output does not leak secrets
```

Build approach

Used by server endpoint. Worker may also expose health depending on deployment model.

platform/workers/

Purpose

Owens background job runtime.

Contents

```
main.ts  
registry.ts  
queue.ts  
scheduler.ts  
processor.ts
```

Example files

```
// platform/workers/registry.ts
export interface JobRegistry {
  register(job: JobDefinition): void;
}
```

```
// platform/workers/processor.ts
export async function processJob(job: RegisteredJob, payload: unknown): Promise<void> {
  // validate payload, create job context, run handler, log outcome
}
```

Allowed dependencies

```
queue libraries
scheduler libraries
platform/contracts
platform/config
platform/runtime
platform/observability
```

Forbidden dependencies

```
business job implementations except through app registration
ad hoc app imports outside mount composition
HTTP server concerns unless shared through contracts
```

Ownership rules

Platform owns job mechanics.

Apps own job handlers.

Testing approach

Test:

```
job registration
payload validation
retry policy
dead-letter behavior
job logging
graceful worker shutdown
duplicate job names fail
```

Build approach

Build as a separate executable:

```
dist/platform/workers/main.js
```

The worker process mounts the same apps but only runs registered jobs.

platform/testing/

Purpose

Provides fakes and helpers for platform/app contract tests.

Contents

```
fake-registry.ts  
fake-context.ts  
test-server.ts  
mount-contract.ts
```

Example files

```
// platform/testing/mount-contract.ts  
export async function expectValidAppMount(app: PlatformApp): Promise<void> {  
  // mount app into fake registry and validate contributions  
}
```

Allowed dependencies

```
test libraries  
platform/contracts  
platform/server test adapters  
in-memory queue/cache fakes
```

Forbidden dependencies

```
production secrets  
real external services by default  
cloud IaC
```

Ownership rules

Platform owns reusable test helpers. App teams use them to verify app mounts.

Testing approach

This folder exists to support tests. It should itself be tested lightly, mostly through consumers.

Build approach

Export separately:

```
@kanbien/platform/testing
```

Do not include test helpers in production bundles.

The business app mounting contract

The key design decision:

Apps register contributions. They do not control the server.

A business app should export a single mount module:

```
apps/crm/app.mount.ts
```

That module should be the app's public backend integration point.

It registers:

```
routes
permissions
jobs
health checks
config schema if needed
lifecycle hooks if needed
```

The platform imports only that file.

Example platform contract

Here is a simplified but realistic TypeScript contract.

```
// platform/contracts/app.ts

export type ApiVersion = 'v1' | 'v2';

export type HttpMethod =
  | 'GET'
  | 'POST'
  | 'PUT'
  | 'PATCH'
  | 'DELETE';

export interface PlatformApp {
  name: string;
  basePath: `/${string}`;

  mount(
    registry: AppRegistry,
    deps: AppMountDeps
  ): void | Promise<void>;
}

export interface AppMountDeps {
  logger: Logger;
  config: AppConfigReader;
  resources: RuntimeResources;
}

export interface AppRegistry {
  routes: RouteRegistry;
  permissions: PermissionRegistry;
  jobs: JobRegistry;
  health: HealthRegistry;
}

export interface RouteRegistry {
  register(route: RouteDefinition): void;
}

export interface PermissionRegistry {
  define(permission: PermissionDefinition): void;
  defineMany(permissions: PermissionDefinition[]): void;
}

export interface JobRegistry {
  register(job: JobDefinition): void;
}

export interface HealthRegistry {
  register(check: HealthCheckDefinition): void;
}

export interface RouteDefinition {
  method: HttpMethod;
  version: ApiVersion;
  path: `/${string}`;
  summary?: string;

  auth: AuthPolicy;
  permissions?: string[];
  rateLimit?: RateLimitPolicy;

  handler: RouteHandler;
}
```



```

export type AuthPolicy =
  | { kind: 'public' }
  | { kind: 'required' };

export interface RateLimitPolicy {
  policy: 'standard' | 'sensitive' | 'public';
  key?: 'ip' | 'principal' | 'tenant';
}

export interface RouteHandler {
  (
    ctx: RequestContext,
    request: AppRequest
  ): Promise<AppResponse> | AppResponse;
}

export interface AppRequest {
  params: Record<string, string>;
  query: Record<string, string | string[] | undefined>;
  body: unknown;
  headers: Record<string, string | undefined>;
}

export interface AppResponse {
  status: number;
  body?: unknown;
  headers?: Record<string, string>;
}

export interface PermissionDefinition {
  id: string;
  description: string;
}

export interface JobDefinition {
  name: string;
  queue: string;
  description?: string;
  payloadSchema?: Schema<unknown>;
  retry?: {
    attempts: number;
    backoffMs: number;
  };
  schedule?: {
    cron: string;
  };
  handler: JobHandler;
}

export interface JobHandler {
  (
    ctx: JobContext,
    payload: unknown
  ): Promise<void>;
}

export interface HealthCheckDefinition {
  name: string;
  kind: 'liveness' | 'readiness';
  critical: boolean;
  check: HealthCheck;
}

export interface HealthCheck {
  (): Promise<HealthCheckResult>;
}

export interface HealthCheckResult {
  ok: boolean;
  message?: string;
  details?: Record<string, unknown>;
}

```

```

export interface RequestContext {
  requestId: string;
  correlationId: string;
  tenantId?: string;
  principal?: Principal;
  logger: Logger;
  flags: FeatureFlagReader;
  config: AppConfigReader;
  now: () => Date;
  abortSignal?: AbortSignal;
}

export interface JobContext {
  jobId: string;
  jobName: string;
  tenantId?: string;
  logger: Logger;
  flags: FeatureFlagReader;
  config: AppConfigReader;
  resources: RuntimeResources;
  now: () => Date;
}

export interface Principal {
  id: string;
  tenantId?: string;
  roles: string[];
  permissions: string[];
}

export interface Logger {
  info(fields: Record<string, unknown>, message: string): void;
  warn(fields: Record<string, unknown>, message: string): void;
  error(fields: Record<string, unknown>, message: string): void;
  child(fields: Record<string, unknown>): Logger;
}

export interface FeatureFlagReader {
  isEnabled(flag: string): Promise<boolean>;
}

export interface AppConfigReader {
  getString(key: string): string;
  getNumber(key: string): number;
  getBoolean(key: string): boolean;
  optionalString(key: string): string | undefined;
}

export interface RuntimeResources {
  db?: unknown;
  cache?: unknown;
  queue?: unknown;
}

export interface Schema<T> {
  parse(input: unknown): T;
}

```

This contract is intentionally boring. That is good.

It gives apps enough power to plug in, but not enough power to hijack the runtime.

Worked example: apps/crm/app.mount.ts

Assume CRM has users.

```
apps/  
  crm/  
    app.mount.ts  
    users/  
      user.routes.ts  
      user.permissions.ts  
      user.jobs.ts  
      user.health.ts  
      user.service.ts  
      user.repository.ts
```

The mount file:

```
// apps/crm/app.mount.ts  
  
import type { PlatformApp } from '../../platform/contracts/app';  
  
import {  
  listUsers,  
  getUser,  
  createUser,  
  updateUser,  
  deactivateUser  
} from './users/user.routes';  
  
import { userPermissions } from './users/user.permissions';  
  
import {  
  syncUsersFromIdentity,  
  rebuildUserSearchIndex  
} from './users/user.jobs';  
  
import {  
  checkUserRepositoryHealth,  
  checkUserSearchHealth  
} from './users/user.health';  
  
export const crmApp: PlatformApp = {  
  name: 'crm',  
  basePath: '/crm',  
  
  async mount(registry, deps) {  
    const logger = deps.logger.child({ app: 'crm' });  
  
    logger.info({}, 'mounting crm app');  
  
    /**  
     * 1. User permissions  
     */  
    registry.permissions.defineMany(userPermissions);  
  
    /**  
     * 2. User routes  
     *  
     * These are local CRM paths.  
     * The platform will expose them as:  
     *  
     *   /api/v1/crm/users  
     *   /api/v1/crm/users/:userId  
     */  
    registry.routes.register({  
      method: 'GET',
```

```

    version: 'v1',
    path: '/users',
    summary: 'List CRM users',
    auth: { kind: 'required' },
    permissions: ['crm.users.read'],
    rateLimit: {
      policy: 'standard',
      key: 'principal'
    },
    handler: listUsers
  });

registry.routes.register({
  method: 'GET',
  version: 'v1',
  path: '/users/:userId',
  summary: 'Get CRM user',
  auth: { kind: 'required' },
  permissions: ['crm.users.read'],
  rateLimit: {
    policy: 'standard',
    key: 'principal'
  },
  handler: getUser
});

registry.routes.register({
  method: 'POST',
  version: 'v1',
  path: '/users',
  summary: 'Create CRM user',
  auth: { kind: 'required' },
  permissions: ['crm.users.create'],
  rateLimit: {
    policy: 'sensitive',
    key: 'principal'
  },
  handler: createUser
});

registry.routes.register({
  method: 'PATCH',
  version: 'v1',
  path: '/users/:userId',
  summary: 'Update CRM user',
  auth: { kind: 'required' },
  permissions: ['crm.users.update'],
  rateLimit: {
    policy: 'sensitive',
    key: 'principal'
  },
  handler: updateUser
});

registry.routes.register({
  method: 'POST',
  version: 'v1',
  path: '/users/:userId/deactivate',
  summary: 'Deactivate CRM user',
  auth: { kind: 'required' },
  permissions: ['crm.users.deactivate'],
  rateLimit: {
    policy: 'sensitive',
    key: 'principal'
  },
  handler: deactivateUser
});

/**
 * 3. User background jobs
 */
registry.jobs.register({

```

```

    name: 'crm.users.sync-from-identity',
    queue: 'crm',
    description: 'Synchronize CRM users from the identity system',
    retry: {
      attempts: 5,
      backoffMs: 30_000
    },
    schedule: {
      cron: '*/15 * * * *'
    },
    handler: syncUsersFromIdentity
  });

  registry.jobs.register({
    name: 'crm.users.rebuild-search-index',
    queue: 'crm',
    description: 'Rebuild the CRM user search index',
    retry: {
      attempts: 3,
      backoffMs: 60_000
    },
    handler: rebuildUserSearchIndex
  });

  /**
   * 4. User health checks
   */
  registry.health.register({
    name: 'crm.users.repository',
    kind: 'readiness',
    critical: true,
    check: checkUserRepositoryHealth
  });

  registry.health.register({
    name: 'crm.users.search',
    kind: 'readiness',
    critical: false,
    check: checkUserSearchHealth
  });

  logger.info({}, 'crm app mounted');
}
};

```

Notice what this file does **not** do.

It does not start the server.

It does not install global middleware.

It does not configure CORS.

It does not parse JWTs.

It does not create process signal handlers.

It contributes CRM capabilities to the platform.

Example CRM permission file

```
// apps/crm/users/user.permissions.ts

import type { PermissionDefinition } from '../../../../platform/contracts/app';

export const userPermissions: PermissionDefinition[] = [
  {
    id: 'crm.users.read',
    description: 'Read CRM users'
  },
  {
    id: 'crm.users.create',
    description: 'Create CRM users'
  },
  {
    id: 'crm.users.update',
    description: 'Update CRM users'
  },
  {
    id: 'crm.users.deactivate',
    description: 'Deactivate CRM users'
  }
];
```

The CRM app owns the permission names because they are CRM business concepts.

The platform owns enforcement mechanics.

Example CRM route handler

```
// apps/crm/users/user.routes.ts

import type {
  AppRequest,
  AppResponse,
  RequestContext
} from '../../../platform/contracts/app';

import { createUserService } from './user.service';
import { notFound, forbidden } from '../../../platform/runtime/errors';

export async function listUsers(
  ctx: RequestContext,
  request: AppRequest
): Promise<AppResponse> {
  const userService = createUserService();

  const result = await userService.listUsers({
    tenantId: ctx.tenantId,
    principalId: ctx.principal?.id,
    query: request.query
  });

  return {
    status: 200,
    body: {
      users: result.users
    }
  };
}

export async function getUser(
  ctx: RequestContext,
  request: AppRequest
): Promise<AppResponse> {
  const userService = createUserService();

  const user = await userService.getUser({
    tenantId: ctx.tenantId,
    userId: request.params.userId,
    principalId: ctx.principal?.id
  });

  if (!user) {
    throw notFound('CRM user not found');
  }

  return {
    status: 200,
    body: {
      user
    }
  };
}

export async function createUser(
  ctx: RequestContext,
  request: AppRequest
): Promise<AppResponse> {
  const userService = createUserService();

  const created = await userService.createUser({
    tenantId: ctx.tenantId,
    principalId: ctx.principal?.id,
    input: request.body
  });
}
```

```

    ctx.logger.info(
      { userId: created.id },
      'crm user created'
    );

    return {
      status: 201,
      body: {
        user: created
      }
    };
  }
}

export async function updateUser(
  ctx: RequestContext,
  request: AppRequest
): Promise<AppResponse> {
  const userService = createUserService();

  const updated = await userService.updateUser({
    tenantId: ctx.tenantId,
    userId: request.params.userId,
    principalId: ctx.principal?.id,
    input: request.body
  });

  return {
    status: 200,
    body: {
      user: updated
    }
  };
}

export async function deactivateUser(
  ctx: RequestContext,
  request: AppRequest
): Promise<AppResponse> {
  const userService = createUserService();

  const canDeactivate = await userService.canDeactivateUser({
    tenantId: ctx.tenantId,
    userId: request.params.userId,
    principalId: ctx.principal?.id
  });

  if (!canDeactivate) {
    throw forbidden('You cannot deactivate this CRM user');
  }

  await userService.deactivateUser({
    tenantId: ctx.tenantId,
    userId: request.params.userId,
    principalId: ctx.principal?.id
  });

  return {
    status: 204
  };
}

```

This shows the split clearly:

```

Platform:
  route auth and permission check

CRM app:
  actual CRM user rules

```


Example CRM jobs

```
// apps/crm/users/user.jobs.ts

import type { JobContext } from '../../../platform/contracts/app';

import { createUserSyncService } from './user-sync.service';
import { createUserSearchService } from './user-search.service';

export async function syncUsersFromIdentity(
  ctx: JobContext,
  payload: unknown
): Promise<void> {
  const service = createUserSyncService();

  ctx.logger.info({}, 'starting crm user identity sync');

  await service.syncFromIdentity({
    tenantId: ctx.tenantId
  });

  ctx.logger.info({}, 'finished crm user identity sync');
}

export async function rebuildUserSearchIndex(
  ctx: JobContext,
  payload: unknown
): Promise<void> {
  const service = createUserSearchService();

  ctx.logger.info({}, 'rebuilding crm user search index');

  await service.rebuildIndex({
    tenantId: ctx.tenantId
  });

  ctx.logger.info({}, 'rebuilt crm user search index');
}
```

The platform handles queue mechanics.

CRM handles what the job actually does.

Example CRM health checks

```
// apps/crm/users/user.health.ts

import type {
  HealthCheckResult
} from '../../../platform/contracts/app';

import { createUserRepository } from './user.repository';
import { createUserSearchService } from './user-search.service';

export async function checkUserRepositoryHealth(): Promise<HealthCheckResult> {
  const repository = createUserRepository();

  const ok = await repository.canConnect();

  return {
    ok,
    message: ok
      ? 'CRM user repository is reachable'
      : 'CRM user repository is not reachable'
  };
}

export async function checkUserSearchHealth(): Promise<HealthCheckResult> {
  const search = createUserSearchService();

  const ok = await search.canSearch();

  return {
    ok,
    message: ok
      ? 'CRM user search is reachable'
      : 'CRM user search is not reachable'
  };
}
```

Again, the app owns the meaning. The platform owns aggregation and exposure.

platform/server/mount.ts

The platform composition root imports app mounts and registers them.

```
// platform/server/mount.ts

import type {
  AppRegistry,
  PlatformApp
} from '../contracts/app';

import { createPlatformRegistry } from './registry';

import { crmApp } from '../../apps/crm/app.mount';
// Later:
// import { billingApp } from '../../apps/billing/app.mount';
// import { adminApp } from '../../apps/admin/app.mount';

export interface PlatformRuntime {
  logger: {
    child(fields: Record<string, unknown>): any;
    info(fields: Record<string, unknown>, message: string): void;
  };
  config: any;
  resources: any;
}

export interface MountedPlatform {
  registry: AppRegistry;
}

const installedApps: PlatformApp[] = [
  crmApp
  // billingApp,
  // adminApp
];

export async function mountApps(
  runtime: PlatformRuntime
): Promise<MountedPlatform> {
  const registry = createPlatformRegistry();

  for (const app of installedApps) {
    validateAppIdentity(app);

    const appLogger = runtime.logger.child({
      app: app.name
    });

    const scopedRegistry = registry.scopeToApp({
      appName: app.name,
      basePath: app.basePath
    });

    runtime.logger.info(
      { app: app.name },
      'mounting app'
    );

    await app.mount(scopedRegistry, {
      logger: appLogger,
      config: runtime.config.forApp?.(app.name) ?? runtime.config,
      resources: runtime.resources
    });

    runtime.logger.info(
      { app: app.name },
      'mounted app'
    );
  }
}
```

```

    });
  }

  registry.validate();

  return {
    registry
  };
}

function validateAppIdentity(app: PlatformApp): void {
  if (!app.name || !/^[a-z][a-z0-9-]*$/i.test(app.name)) {
    throw new Error(`Invalid platform app name: ${app.name}`);
  }

  if (!app.basePath.startsWith('/')) {
    throw new Error(`Invalid basePath for app ${app.name}`);
  }

  if (app.basePath === '/') {
    throw new Error(`App ${app.name} cannot mount at root`);
  }
}

```

The scoped registry should automatically prefix app routes.

If CRM registers:

```
GET /users
```

The platform exposes:

```
GET /api/v1/crm/users
```

That prevents apps from colliding with each other.

platform/server/registry.ts

The registry is where the platform validates app contributions.

```
// platform/server/registry.ts

import type {
  AppRegistry,
  HealthCheckDefinition,
  JobDefinition,
  PermissionDefinition,
  RouteDefinition
} from '../contracts/app';

type ScopedRegistryInput = {
  appName: string;
  basePath: `/${string}`;
};

export function createPlatformRegistry() {
  const routes: RouteDefinition[] = [];
  const permissions: PermissionDefinition[] = [];
  const jobs: JobDefinition[] = [];
  const healthChecks: HealthCheckDefinition[] = [];

  return {
    scopeToApp(input: ScopedRegistryInput): AppRegistry {
      return {
        routes: {
          register(route) {
            routes.push({
              ...route,
              path: joinPaths(input.basePath, route.path)
            });
          }
        },
        permissions: {
          define(permission) {
            permissions.push(permission);
          },
          defineMany(items) {
            for (const item of items) {
              permissions.push(item);
            }
          }
        },
        jobs: {
          register(job) {
            jobs.push({
              ...job,
              name: namespaceJobName(input.appName, job.name)
            });
          }
        },
        health: {
          register(check) {
            healthChecks.push({
              ...check,
              name: namespaceHealthName(input.appName, check.name)
            });
          }
        }
      };
    },
  },
}
```

```

    validate() {
        assertNoDuplicateRoutes(routes);
        assertNoDuplicatePermissions(permissions);
        assertNoDuplicateJobs(jobs);
        assertNoDuplicateHealthChecks(healthChecks);
        assertRoutesReferenceKnownPermissions(routes, permissions);
    },

    get routes() {
        return routes;
    },

    get permissions() {
        return permissions;
    },

    get jobs() {
        return jobs;
    },

    get healthChecks() {
        return healthChecks;
    }
};

function joinPaths(left: string, right: string): `/${string}` {
    return `${left.replace(/\$/ /, ' ')}${right.replace(/^\$/ /, ' ')} as `/${string}`;
}

function namespaceJobName(appName: string, jobName: string): string {
    if (jobName.startsWith(`${appName}.`)) {
        return jobName;
    }

    return `${appName}.${jobName}`;
}

function namespaceHealthName(appName: string, checkName: string): string {
    if (checkName.startsWith(`${appName}.`)) {
        return checkName;
    }

    return `${appName}.${checkName}`;
}

function assertNoDuplicateRoutes(routes: RouteDefinition[]): void {
    const seen = new Set<string>();

    for (const route of routes) {
        const key = `${route.method} ${route.version} ${route.path}`;

        if (seen.has(key)) {
            throw new Error(`Duplicate route registered: ${key}`);
        }

        seen.add(key);
    }
}

function assertNoDuplicatePermissions(permissions: PermissionDefinition[]): void {
    const seen = new Set<string>();

    for (const permission of permissions) {
        if (seen.has(permission.id)) {
            throw new Error(`Duplicate permission registered: ${permission.id}`);
        }

        seen.add(permission.id);
    }
}

```

```

function assertNoDuplicateJobs(jobs: JobDefinition[]): void {
  const seen = new Set<string>();

  for (const job of jobs) {
    if (seen.has(job.name)) {
      throw new Error(`Duplicate job registered: ${job.name}`);
    }

    seen.add(job.name);
  }
}

function assertNoDuplicateHealthChecks(checks: HealthCheckDefinition[]): void {
  const seen = new Set<string>();

  for (const check of checks) {
    if (seen.has(check.name)) {
      throw new Error(`Duplicate health check registered: ${check.name}`);
    }

    seen.add(check.name);
  }
}

function assertRoutesReferenceKnownPermissions(
  routes: RouteDefinition[],
  permissions: PermissionDefinition[]
): void {
  const knownPermissionIds = new Set(
    permissions.map(permission => permission.id)
  );

  for (const route of routes) {
    for (const permission of route.permissions ?? []) {
      if (!knownPermissionIds.has(permission)) {
        throw new Error(
          `Route ${route.method} ${route.path} references unknown permission ${permission}`
        );
      }
    }
  }
}

```

The real implementation will be more polished, but this shows the shape.

The platform validates the system before it starts serving traffic. That is exactly what you want.

How route registration becomes real HTTP routes

Your app registers this:

```
registry.routes.register({
  method: 'GET',
  version: 'v1',
  path: '/users',
  auth: { kind: 'required' },
  permissions: ['crm.users.read'],
  handler: listUsers
});
```

The platform turns it into:

```
GET /api/v1/crm/users
```

And wraps the handler with:

```
request context creation
rate limiting
authentication
authorization
validation
logging
error mapping
```

Conceptually:

```
// platform/server/routes.ts

export function registerAppRoutes(server: HttpServer, registry: MountedRegistry) {
  for (const route of registry.routes) {
    const fullPath = `/api/${route.version}${route.path}`;

    server.register({
      method: route.method,
      path: fullPath,

      async handler(rawRequest, rawResponse) {
        const ctx = await createRequestContextFromRawRequest(rawRequest);

        await enforceRateLimit(route, ctx, rawRequest);
        await authenticateIfRequired(route, ctx, rawRequest);
        await authorizeIfRequired(route, ctx);

        try {
          const response = await route.handler(ctx, {
            params: rawRequest.params,
            query: rawRequest.query,
            body: rawRequest.body,
            headers: rawRequest.headers
          });
        }

        return rawResponse
          .status(response.status)
          .headers(response.headers ?? {})
          .send(response.body);
      }
    });
  }
}
```



```
        } catch (error) {  
            return handleError(error, ctx, rawResponse);  
        }  
    }  
});  
}  
}
```

The app handler stays simple.

The platform owns the ceremony.

What should remain inside `apps/crm`

Keep CRM-specific behavior inside `apps/crm`.

```
apps/crm/  
  app.mount.ts  
  
  users/  
    user.routes.ts  
    user.controller.ts  
    user.service.ts  
    user.repository.ts  
    user.permissions.ts  
    user.jobs.ts  
    user.health.ts  
    user.types.ts  
    user.validation.ts  
  
  accounts/  
    account.routes.ts  
    account.service.ts  
    account.repository.ts  
  
  contacts/  
    contact.routes.ts  
    contact.service.ts  
    contact.repository.ts  
  
  pipelines/  
    pipeline.service.ts  
    pipeline.rules.ts  
  
  db/  
    migrations/  
      schema.ts  
  
  tests/  
    crm.mount.test.ts  
    users/  
      user.service.test.ts
```

CRM owns:

```
CRM domain models  
CRM use cases  
CRM route handlers  
CRM validation  
CRM repository logic  
CRM database schema/migrations  
CRM permissions  
CRM background job handlers  
CRM health check implementations  
CRM-specific config names  
CRM-specific feature flag names  
CRM-specific authorization after resource loading
```

Example CRM-specific authorization:

```
"Can this sales manager deactivate this CRM user?"
```

That belongs in CRM.

What should move into `packages/core`

Move code to `packages/core` only when it is stable, framework-independent, and genuinely reused.

Good candidates:

```
packages/core/result
packages/core/errors
packages/core/id
packages/core/time
packages/core/pagination
packages/core/tenant
packages/core/email
packages/core/money
packages/core/validation
```

Examples:

```
// packages/core/result.ts
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };
```

```
// packages/core/pagination.ts
export interface PageRequest {
  limit: number;
  cursor?: string;
}

export interface Page<T> {
  items: T[];
  nextCursor?: string;
}
```

```
// packages/core/tenant.ts
export type TenantId = string & { readonly brand: unique symbol };
```

Do not move CRM concepts into core too early.

Bad candidates:

```
CRMUser
SalesPipeline
LeadStatus
BillingPlan
InvoiceLifecycle
```

Those are not core. They are business domain concepts.

A good rule:

Move code to packages/core after the second or third real consumer, not because you imagine reuse.

What belongs in /platform

These belong in /platform:

- HTTP server startup
- app mounting
- route registry
- middleware ordering
- CORS
- rate limiting
- auth hooks
- request context creation
- standard error mapping
- structured logging
- health endpoint aggregation
- environment config loading
- feature flag access
- API versioning
- background worker runtime
- queue mechanics
- graceful shutdown
- runtime resource lifecycle

These do not belong in /platform:

- CRM user logic
- CRM repository methods
- billing plan rules
- customer lifecycle workflows
- feature-specific jobs
- business-specific health decisions
- domain-specific permissions beyond names/registration

Minimal v1 platform architecture

For Kanbien v1, I would build this:

```
platform/  
  contracts/  
  config/  
  runtime/  
  server/  
  security/  
  observability/  
  health/  
  workers/  
  testing/
```

And I would explicitly **not** build:

```
generic plugin marketplace  
custom dependency injection framework  
multi-transport RPC framework  
custom workflow engine  
custom feature flag system  
custom service mesh abstraction  
generic repository framework  
generic domain event framework  
multi-database abstraction
```

The v1 should have enough structure to be enterprise-ready:

Must-have v1 pieces

```
Typed app mount contract  
Explicit app manifest  
HTTP server endpoint  
Worker endpoint  
Route registry  
Permission registry  
Job registry  
Health registry  
Typed env config  
Request context  
Structured logging  
Standard error responses  
CORS  
Rate limiting  
Auth hook interface  
Feature flag reader interface  
Graceful shutdown  
Contract tests
```

Nice-to-have later

```
OpenAPI generation from route registry  
admin UI for registered routes/jobs/health  
distributed tracing  
advanced metrics dashboards  
dynamic app loading
```

advanced policy engine
multi-region worker routing
full audit event pipeline

Do not start with those unless Kanbien actually needs them.

The simple v1 runtime flow

API process:

```
platform/server/main.ts
load config
create logger
create resources
mount apps
validate registry
create HTTP server
install middleware
register health routes
register app routes
start listening
install graceful shutdown
```

Worker process:

```
platform/workers/main.ts
load config
create logger
create resources
mount apps
validate registry
register job processors
start consuming jobs
install graceful shutdown
```

Same apps. Same contract. Different entrypoint.

That is powerful without being bloated.

The most important design rule

The platform should be strict at the boundary and boring inside.

Business apps should be able to say:

```
export const crmApp: PlatformApp = {  
  name: 'crm',  
  basePath: '/crm',  
  mount(registry, deps) {  
    registry.routes.register(...);  
    registry.permissions.defineMany(...);  
    registry.jobs.register(...);  
    registry.health.register(...);  
  }  
};
```

And the platform should guarantee:

```
the route is namespaced  
the route is versioned  
auth runs consistently  
permissions are checked consistently  
rate limits are applied consistently  
logs are structured  
errors are safe  
health is aggregated  
jobs run with retries  
shutdown is graceful
```

That is the platform's job.

Everything else should have to earn its place.